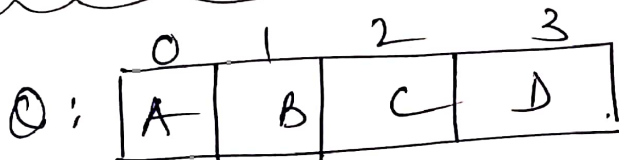


Queue Data Structure / AOT

What is queue?

- ✓ A queue is a linear data structure in which insertion is performed from one end called the Rear, and deletion is performed from the other end called the Front.
- ✓ A queue is a linear data structure that follows FIFO (First-In, First-Out) principle, where the first element inserted is the first element removed.

Queue Illustration



FRONT = 0 REAR = 3

Basic Queue operations

Operation	Description
1. Enqueue	Add an element at the Rear
2. Dequeue	Remove an element from the Front
3. Peek/Front	View the first element w/o removing it
4. isEmpty()	checks whether the queue contains any elements
5. isFull()	checks overflow in array implementation

Types of Queue

There are four major types of queues

- (1) Simple Queue (Linear Queue)
- (2) Circular Queue
- (3) Double-Ended Queue
- (4) Priority Queue.

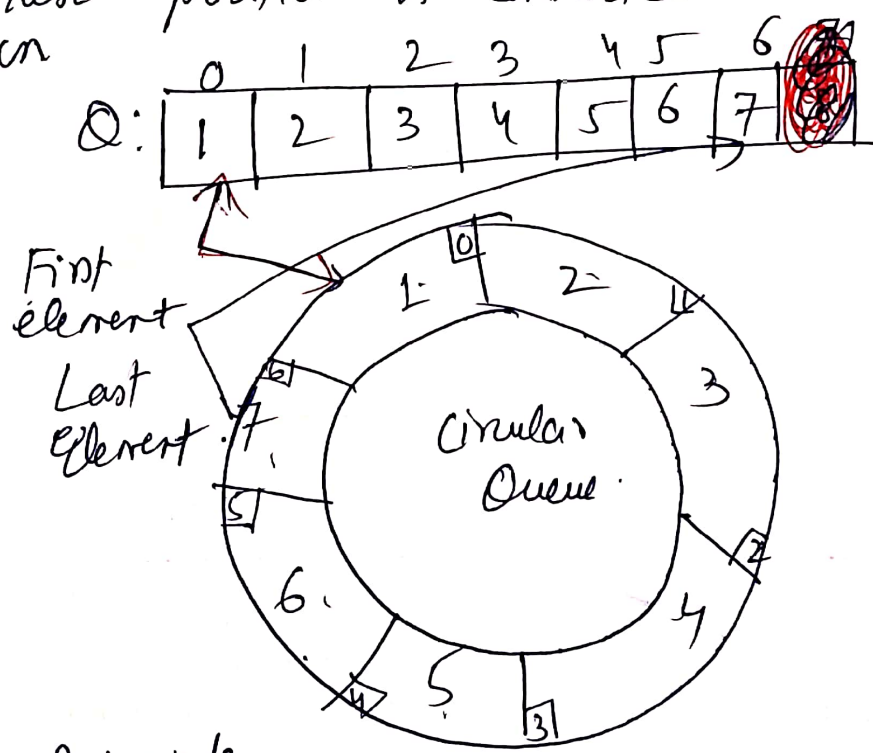
(1) Simple / Linear Queue

— Insertion at Rear & deletion at Front
characteristics :

- (1) FIFO order
- (2) Easy implementation using arrays or linked list
- (3) May waste memory in array implementation after multiple dequeues.

(2) Circular Queue / Ring Buffer

The last position is connected to the first position



Working Principle

1. Front & Rear move circularly
2. Reuses empty spaces
3. Avoids mem wastage

3) priority Queue

Every element has a priority

Types:

- ✓ 1. Ascending PQ - smallest priority served first
- ✓ 2. Descending PQ - highest - - - - -

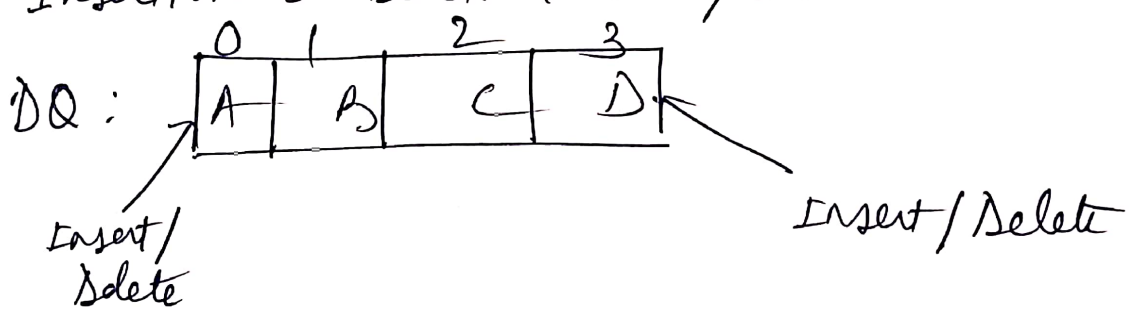
Example:

<u>Process</u>	<u>priority</u>
P1	3
P2	1
P3	2

Execution order: P2 → P3 → P1

4) Double Ended Queue (Deque)

Insertion & Deletion are possible at both ends



#Types of Deque

1. Input-restricted Deque

Insertion at one end
deletion at both ends

2. Output-restricted Deque

Deletion at one end,
insertion at both ends

CS Applic

O/S	CPU scheduling, Interrupt Queue
Networking	Packed Transmission
Compiler	BFS Traversal
Printer	Printer Queue
Bank/Hospital	Customer waiting Queue
Simulation	Traffic & Event Simulation

Stack vs Queue

Stack	Queue
LIFO (Last-In, First-Out)	FIFO (First-In, First-Out)
Insertion & Deletion occur at Top	Insertion at Rear & Deletion from Front
Main pointer: Top	Main pointers: Front & Rear
Operations: push, pop	Operations: enqueue, dequeue
Latest element removed first	Oldest element removed first
Used in recursion, Undo, expr. eval	Used in scheduling, buffering, BFS traversal.

(5)

pseudocode uses an array implementation of a Linear Queue with FRONT & REAR pointers.
Initial values

MAX = size of Queue

QUEUE[MAX]

FRONT = -1

REAR = -1

① Enqueue operation (Insertion)

Purpose: Insert an element at the rear of the queue.

Enqueue (item)

```

{
    if (REAR REAR == MAX - 1)
    {
        print("Queue overflow")
    }
    else
    {
        if (FRONT == -1)
        {
            FRONT = 0
        }
        REAR = REAR + 1
        QUEUE[REAR] = item
        print("item, "inserted successfully")
    }
}

```

Time: $O(1)$

.) Dequeue Operation (Deletion)

purpose: Remove an element from the Front of the queue.

Dequeue()

{

if (FRONT == -1 or FRONT > REAR)

{

print("Queue Underflow")

}

else

{

item = QUEUE[FRONT]

print(item, "deleted successfully")

FRONT = FRONT + 1

if (FRONT > REAR)

{

FRONT = -1

REAR = -1

}

}

}

Time : $O(1)$

3. peek (Front Element)

Purpose: Display the first element w/o deleting it

```
peek()
```

```
{ if (FRONT == -1 or FRONT > REAR)
  { print ("Queue is Empty")
```

```
  } else {
```

```
    print ("Front Element =",
           QUEUE[FRONT])
```

```
  }
}
```

Time Complexity: $O(1)$

4. Display Queue

Purpose: Display all elements from FRONT to REAR.

```
Display()
```

```
{ if (FRONT == -1 or FRONT > REAR)
  {
```

```
  } else {
```

```
    for (i = FRONT; i < REAR; i++)
```

```
    { print (QUEUE[i])
```

```
  }
}
```

Time: $O(n)$

is Empty()

is Empty()

```
{ if (FRONT == -1 or FRONT > REAR)
  { return True
  } else {
    return False
  }
}
```

6. is Full()

```
{ if (REAR == MAX - 1)
  return True.
```

```
else return False
```

```
}
```

Pseudocode Circular Queue / Buffer Ring

A circular queue is a queue in which the last position is connected to the first position. It efficiently reuses empty spaces created after deletion.

Initial Declaration

MAX — size of Queue.

QUEUE[MAX]

FRONT = -1, REAR = -1

Important Conditions.

• Queue is Empty: FRONT = -1

• Queue is Full:

$(REAR + 1) \% MAX == FRONT$

(1) Enqueue Operation (Insertion)

Purpose: Insert an element at the Rear

Enqueue(item)

if (isFull())

{ print("Queue Full")

} else {

if (FRONT == -1) {

FRONT = 0

REAR = 0

else

REAR

isFull()

{ if ((REAR + 1) % MAX == FRONT)

return True

} else {

return False

}

}

}

}

}

}

}

(i) Enqueue (item)

```
{
    if (isFull())
    {
        print("Queue is Full")
    }
    else {
        if (FRONT == -1)           // Empty first time
        {
            FRONT = 0
            REAR = 0
        }
        else {
            REAR = (REAR + 1) % MAX
        }
        QUEUE[REAR] = item
        print(item, "inserted successfully")
    }
}
```

Time : $O(1)$

```
# isFull()
{
    if ((REAR + 1) % MAX == FRONT)
        return True
    else
        return False
}
```

2. Dequeue()

```

{
    if (isEmpty())
    {
        print("Queue is Empty")
    }
    else {
        item = QUEUE[FRONT]
        print(item, "deleted successfully")
        if (FRONT == REAR) // one element
        {
            FRONT = -1
            REAR = -1
        }
        else {
            FRONT = (FRONT + 1) % MAX
        }
    }
}

```

Time = $O(1)$

```

# isEmpty()
{
    if (FRONT == -1)
        return True
    else
        return False
}

```

3. peek()

```
{
  if (isEmpty())
  {
    print("Queue is empty")
  }
  else {
    print("Front Element", QUEUE[FRONT])
  }
}
```

Time: $O(1)$

4. Display: display all elements in the circular queue.

Display()

```
{
  if (isEmpty())
  {
    print( - - - )
  }
  else {
```

```
    i = FRONT
```

```
    while true do
    {
      print(QUEUE[i])
      if (i == REAR)
      {
        break
      }
      i = (i+1) mod MAX
```

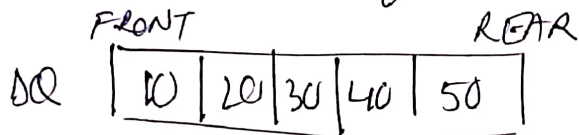
Time = $O(n)$

```
    }
  }
```

Double-Ended Queue

- is a linear d/s in which insertion & deletion are possible from both the front & rear

Defn: A deque is a queue that supports insertion & deletion from both ends of the queue.



characteristics of Deque

- Nature: Linear d/s
- Insertion: Front & Rear
- Deletion: Front & Rear
- Memory: Array or LL
- Traversal: Front to Rear
- Time: $O(1)$ for insertion/deletion

Types of Deque

(1) Input-Restricted Deque

- Insertion allowed only from rear
- Deletion " from Front & Rear

(2) Output-Restricted Deque

- Insertion allowed from both ends
- Deletion " only from the front

Types based on Implementation

1. Linear Deque using Array ✓
2. Circular Deque " Circular array
3. Deque using DLL

① Linear Array Deque

0	1	2	3	4
10	20	30	40	50

Front = 1 Rear = 4

Basic Operations

Op.	Desc
Insert Front	add element before current front
Insert Rear	" " after current rear
Delete Front	Remove first element
Delete Rear	Remove last element
Peek Front	Display front element
Peek Rear	" rear "
Display	Traverse queue from front to rear

Alg. conditions

Linear Deque

Queue is Empty:

Front == -1 and Rear == -1

Queue is Full:

Rear == MAX - 1

Pseudocode for Linear Deque

Initialize Deque

Declare deque[MAX]

Declare front = -1

Declare rear = -1

Initially queue contains no elements

Front = -1 , Rear = -1

Pseudocode : Insert at Front

INS-FRONT(item)

```

{ if front == 0 then
    print "overflow"
else if front == -1 then
    front = 0
    rear = 0
    deque[front] = item
else
    front = front - 1
    deque[front] = item
}

```

Dry Run:

Initial queue :

Deque

0	1	2	3	4
10	20	30	40	

 front = 1 , rear = 3

INS-FRONT(10)

INS-FRONT(20) $\Rightarrow$ "overflow" front = 0 , rear = 3

Pseudocode : Insert at Rear

INS-REAR(item)

```

{ if rear == MAX-1 then
    print "overflow"
else if rear == -1 then
    front = 0
    rear = 0
    deque[rear] = item
}

```

else

$$\text{rear} = \text{rear} + 1$$

$$\text{deque}[\text{rear}] = \text{item}$$

}

- First insertion creates queue
- Otherwise rear moves right

Deque:

0	1	2	3	4
	20	30	40	50

INS-REAR(50)

front = 1, rear = 3

front = 1, rear = 4

INS-REAR(60) "overflow"

Pseudocode: Delete from Front

DEL-FRONT()

{

if front == -1 then
print "Underflow"

#empty queue.

else

item = deque[front]

if front == rear then #end element

{ front = -1

rear = -1

}

else

{ front = front + 1

#Multiple

element

}

return item

}

3 cases:

- (1) Empty Queue
- (2) One element
- (3) Multiple elements

Deque:

0	1	2	3
10	20	30	40

front = 0, rear = 3

DEL-FRONT() item = 10

front = 1, rear = 3

Front shifts left

Pseudocode : Delete from Rear

DEL-REAR()

```

{ if rear == -1 then {
    print "Underflow"
} else {
    item = deque[rear]
    if front == rear then {
        front = -1
        rear = -1
    } else {
        rear = rear - 1
    }
}

```

Rear moves one position left.

Deque

0	1	2	3
10	20	30	40

DEL-REAR() front=0, rear=3

item=40 front=0, rear=2

Pseudocode : Peek Front

PEEK-FRONT()

```

{ If front == -1 then {
    print "Queue Empty"
} else {
    print deque[Front]
}
}

```

Pseudocode : Peek Rear

PEEK-REAR()

```

{ if rear == -1 { print " " } else {
    print deque[rear]
}
}

```

Pseudocode : Display

```

DISPLAY()
{
    if front == -1 {
        print("Queue Empty")
    } else {
        for (i = front; i < rear; i++)
        {
            print(queue[i])
        }
    }
}

```

Adv. of Deque

- Insert & Delete from both ends
- Faster than shifting elements in arrays
- Useful in sliding window problems
- Can behave as both stack & queue
- Efficient $O(1)$ insertion/deletion

Disadv

- Indices before front cannot be reused
- This problem is solved using Circular Deque.

Applications of Deque

Appli	How Deque is Used
Browser History	Back & Forward Navigation
Undo/Redo	Insert/Delete from ends
Task Scheduling	Higher priority tasks at front
Palindrome	
Sliding Window Max	Maintain max ^m efficiently
Music Playlist	Next/Prev songs

Seq:

0	1	2	3	4
40	20	30		

front = -1, rear = -1 MAX = 5

(1) INS-FRONT(10) front=0, rear=0

(2) INS-FRONT(20) "overflow"

(3) DEL-FRONT() item=10, front=-1, rear=-1

(4) INS-REAR(10) front=0, rear=0

(5) INS-REAR(20) front=0, rear=1

(6) INS-REAR(30) front=0, rear=2

(7) DEL-FRONT() item=10 front=1, rear=2

(8) INS-FRONT(40) front=0, rear=2

(9) INS-FRONT(50) "overflow".

Priority Queue

A priority queue is a queue in which each element has a priority. Elements with higher priority are deleted first, regardless of their insertion order.

Note: Smaller priority number = Higher Priority
(Priority 1 is served before Priority 2)

Initial Declaration

MAX = size of Queue

QUEUE[MAX], PRIORITY[MAX]

FRONT = -1, REAR = -1

(i) Enqueue: (Insertion with priority)

Purpose: Insert an element acc. to its priority.

Enqueue(item, p)

{ if (REAR == MAX - 1) { ... }

else {

if (FRONT == -1)

{ FRONT = 0, REAR = 0

} else {

REAR = REAR + 1

while $i^c > \text{FRONT}$ and

PRIORITY[i-1] > p do

QUEUE[i] = QUEUE[i-1]

PRIORITY[i] = PRIORITY[i-1]

i = i - 1

j { QUEUE[i] = item, PRIORITY[i] = p

Time = $O(n)$

Dequeue()

```

{ if (FRONT == -1) { - - - }
  else {
    if (FRONT == REAR)
      item = QUEUE[FRONT]
      print(item, "deleted successfully")
      if (FRONT == REAR)
      {
        FRONT = -1, REAR = -1
      } else {
        FRONT = FRONT + 1
      }
    }
  }
}

```

Time = O(1)

Example

Operation

Enqueue(A, 3)

Queue

A(3)

FRONT = -1 MAX = 5
REAR = -1

Queue

0	1	2	3	4
	A	A	A	

Priority

1	2	3	4

FRONT = 0, REAR = 0
I = 0

Enqueue(B, 1)

FRONT = 0 REAR = 1
I = 1

Enqueue(C, 2)

FRONT = 0, REAR = 2
I = 2 > 0 and
I = 1 > 0

Enqueue(D, 4)

FRONT = 0, REAR = 3
I = 3 > 0

0

Priority Queues using Array -

- Understand the concept of a PQ
- Differentiate between Queue & PQ
- Explain Types of PQ
- Implement PQ using an array
- Insertion & Delⁿ ops.
- Analyse TC
- Numericals & prog'ing problems

PQ:

- a special type of queue where every element has a priority associated with it
- unlike normal queue (FIFO), deletion depends on priority rather than insertion order

Example

Patients in a hospital emergency room

Patient	Priority
Critical Patient	1 (Highest)
Accident Patient	2
Fever Patient	5 (Lowest)

Selection order

Critical → Accident → Fever

②

Queue vs PQ

Queue	PQ
FIFO	Priority-based removal
First inserted remove first	Highest priority removed first
One front & one rear	priority determines front element
Simple Scheduling	Emergency scheduling CPU scheduling

Types of PQ

(1) Ascending PQ
smaller number means higher priority.

Element	priority
A	4
B	1
C	3

Deletion order: $B \rightarrow C \rightarrow A$

(2) Descending PQ
Larger number means higher priority

Element	priority
A	4
B	1
C	5

Deletion order: $C \rightarrow A \rightarrow B$



3

Representation using Arrays

We use two arrays.

Array	Purpose
item[]	stores data.
priority[]	stores priority values.

Example:

Item Array

A	B	C	
---	---	---	--

Priority Array

3	1	2	
---	---	---	--

Operations of PQ

- (1) Insertion — insert element with priority
- (2) Deletion — remove highest-priority element
- (3) Peek — View highest-priority element
- (4) isEmpty / isFull — check queue status

Insert into PQ (Ascending)

Logic

- (1) Check overflow
- (2) Insert element at rear
- (3) Insert priority
- (4) Increase rear

4

Pseudocode

```
INSERT - APO(item, priority)
{
    if (rear == MAX)
    {
        print("Queue overflow")
    }
    else {
        rear = rear + 1
        data[rear] = item
        prio[rear] = priority
    }
}
```

Dry Run:

```
INSERT - APO(A, 3)
INSERT
INSERT - APO(B, 1)
```

Index	Data	Priority
0	A	3
1	B	1

Delete Highest Priority Element

Logic
Find Minimum priority

```
DELETE - APO()
```

```
{
    if (rear == -1) {
        print "Queue underflow"
    }
    else {
```

highest = 0

~~for all i in 0 to rear~~

5

```
for (i = 1; i < rear; i++)  
{  
    if (prior[i] < prior[highest])  
    {  
        highest = i  
    }  
    print(data[highest])  
}  
for (i = highest; i < rear - 1; i++)  
{  
    data[i] = data[i + 1]  
    prior[i] = prior[i + 1]  
}  
rear = rear - 1
```

3
Dry Run:
Queue

Element	Priority
A	3
B	1
C	2

Highest priority is B

After deletion

Element	Priority
A	3
C	2

Peek Operation

PEEK()

```
{ if (rear == -1)
  { print("Queue is empty")
  }
  else {
```

highest = 0

```
for (i = 1; i < rear; i++)
```

```
{
  if (prior[i] < prior[highest])
  {
    highest = i
  }
}
```

```
print(data[highest])
}
```

Display operation

DISPLAY()

```
{ if (rear == -1)
  {
    . . .
  }
}
```

else {

```
for (i = 1; i < rear; i++)
```

```
{
  print(data[i], prior[i])
}
```

```
}
```

(6) (7)

INSERT - APQ(P, 4)
 INSERT - APQ(Q, 2)
 INSERT - APQ(R, 1)
 INSERT - APQ(S, 5)

data

	1	2	3	4
P	Q	R	S	

~~priority~~ priority

4	2	1	5	
---	---	---	---	--

DELETE - APQ() ~~rear~~ rear = 3

highest = 0 $\text{prior}[\text{highest}] = 4$ $i = 4$
 $\text{prior}[1] = 2$ $2 < 4$

highest = 1
 $\text{prior}[2] < \text{prior}[1]$ $i = 2$

highest = 2

$\text{prior}[3]$ $\text{prior}[2]$ $i = 3$
 5 4 1

data[2] = R

rear = rear - 1 rear = 2

highest = 0

DELETE - APQ $\text{prior}[1]$ $\text{prior}[0]$
 2 4

highest = 1

$\text{prior}[2]$ $\text{prior}[1]$ $i = 2$

data 0 1 2 3 4 $i = 2$ to 3

data

	0	1	2	3	4
P	Q	S			

prio

4	2	5	1		
---	---	---	---	--	--

0

Applications of PQ

<u>Application</u>	<u>Description</u>
<u>CPU Scheduling</u>	<u>Highest priority process executes first</u>
<u>Hospital Emergency</u>	<u>Critical patients treated first</u>
<u>Printer Scheduling</u>	<u>High-priority print jobs first</u>
<u>Network Routing</u>	<u>Important packets transmitted first</u>
<u>Dijkstra's Alg.</u>	<u>Select minimum-distance node</u>

Advantages

- Supports priority-based processing
- Useful in scheduling algos
- Simple implementation using arrays

- Easy to understand for beginners

Disadvantages

- Selection requires searching $O(n)$
- Shifting elements increases execution time
- Not efficient for large datasets